

Chapter 34: Micro Frontends

1. Overview

A micro frontend architecture decomposes a single large frontend application into smaller, independently built, tested, and deployed pieces that are composed together at build time or run time into one cohesive user experience. It is the frontend analogue of microservices: instead of one backend monolith, you avoid one frontend monolith.

The motivation is almost never technical purity — it is organizational. Once a frontend codebase grows large enough that multiple teams are committing to it, you run into:

- **Deployment coupling:** any team's change requires a full-app rebuild/retest/release, so release cadence is bottlenecked by the slowest or riskiest team.
- **Codebase contention:** shared components, shared state, shared routing config become merge-conflict hotspots.
- **Technology lock-in:** the whole app is stuck on one framework/version, even if one team wants to modernize (e.g., migrate a section to a newer Angular major version) without a big-bang rewrite.
- **Ownership blur:** a "vertical slice" (e.g., "Billing", "Onboarding", "Inventory") is owned end-to-end by one team, but in a monolith the code for that slice is interleaved with everyone else's.

Micro frontends let each team own a vertical slice — its own repo, its own CI/CD pipeline, its own release schedule, potentially its own framework version — while a **shell** (a.k.a. **container** or **host**) application composes the slices into one page or one navigation flow for the end user, who should never perceive the seams.

The cost of this freedom is real: duplicated bundles, styling collisions, harder cross-cutting concerns (auth, analytics, error handling), and a more complex build/deploy pipeline. Micro frontends are an organizational-scaling tool, not a default architecture — they should be adopted when team-autonomy pain is real, not because "microservices" sounds modern.

2. Core Concepts

2.1 Why Micro Frontends — the decision drivers

Adopt micro frontends when:

- Multiple autonomous teams must ship to the same product independently, on different schedules.
- Different parts of the product have genuinely different technical needs (one team needs a rewrite, another is stable legacy).
- You need incremental modernization — strangling a legacy AngularJS/jQuery app by replacing sections one at a time with Angular, rather than a risky rewrite.
- Vertical ownership (one team = one business capability, full stack) is a stated org goal (Conway's Law: system architecture mirrors communication structure).

Avoid micro frontends when:

- A single team (or a few tightly coordinated teams) owns the whole frontend — the coordination overhead of micro frontends will simply exceed the coordination overhead you were trying to remove.

- The app is small/medium-sized — module boundaries via Nx/lerna monorepo + lazy-loaded Angular feature modules already solve most of the "one giant bundle" and "merge conflict" problems without the runtime cost.

2.2 Architectural Approaches

There are four broad integration strategies. They differ mainly in **when** composition happens (build time vs. run time) and **how much runtime isolation** each micro frontend gets.

A. Build-Time Integration (npm/library packages)

Each micro frontend is published as a versioned npm package (an Angular library, typically built with `ng-packagr` or Nx). The shell app declares these packages as `dependencies` in its `package.json` and imports their modules/components directly, exactly like any third-party library.

```
shell-app/package.json
{
  "dependencies": {
    "@org/billing-mfe": "^2.3.0",
    "@org/onboarding-mfe": "^1.7.1"
  }
}
```

Pros

- Simplest mental model — it's just Angular library consumption; full type-checking across the boundary, tree-shaking, no runtime resolution surprises.
- Single compiled bundle → easy to reason about performance, no duplicate framework code.
- Works with Angular's Ivy library format out of the box; no special runtime tooling needed.

Cons

- Not truly independently *deployable* — a micro frontend release still requires the shell app to bump its dependency version, rebuild, retest, and redeploy the whole shell. This reintroduces the "everyone waits on shell release" bottleneck, just one layer removed.
- All micro frontends must agree on the same Angular version (and often the same major version of every shared library) because they're compiled into one bundle.
- Slowest feedback loop: publish → bump version → shell CI → shell deploy.

This approach is really "shared library architecture," and many teams that say they want "micro frontends" actually get 90% of the benefit from this simpler pattern with a monorepo (Nx) instead.

B. Run-Time Integration via Module Federation

Introduced by Webpack 5 (and available in `@angular-architects/module-federation` for Angular, which also works with the newer `esbuild`/Vite-based builders via community adapters), Module Federation lets independently built and deployed JavaScript bundles load each other's code **at runtime**, over the network, without a shared compile step.

Key vocabulary:

- **Host (shell/container):** the app the user loads first; it dynamically fetches and instantiates **remotes**.
- **Remote (micro frontend):** a separately built and deployed bundle that exposes specific modules (e.g.,

a component, a module, a routed entry point) via a `remoteEntry.js` manifest file.

- **Shared dependencies:** libraries (like `@angular/core`, `@angular/common`, `rxjs`) that both host and remote depend on; Module Federation can dedupe these so only one copy loads (see §4).
- **Dynamic remotes:** the host doesn't need to know the remote's URL at build time — it can be resolved at runtime (e.g., from a config service or CDN manifest), which is what enables true independent deployability.

Pros

- True independent deployability: a remote team can push a new build to production and users get it on next page load — no shell rebuild.
- Shared dependency deduplication avoids shipping five copies of Angular/RxJS/Zone.js (when versions are compatible).
- Fine-grained: you can expose a single component, not just "the whole app."
- Works well for "compose several MFEs into one visual page" (e.g., a dashboard with widgets from different teams).

Cons

- Runtime resolution = runtime failure modes: a remote can be down, slow, or ship an incompatible shared-dependency version, and none of this is caught at the host's build/CI time.
- Version-matrix testing becomes a real discipline problem — host v3 + remote v7 might never have been tested together.
- Tooling/debugging is heavier: source maps across federated bundles, breakpoints across origins, and webpack-specific configuration (`ModuleFederationPlugin`) needing to stay in sync conceptually (not literally) across repos.
- CSS/global state isolation is *not* provided by Module Federation itself — you must engineer it (see Edge Cases).

C. iframes

The oldest and most brute-force isolation mechanism: each micro frontend is a separate application rendered inside an `<iframe>`.

Pros

- Maximum isolation: separate JS realm, separate DOM, separate CSS cascade — by construction, one MFE literally cannot leak styles or global variables into another, nor crash the host's JS context.
- Can host frontends written in *entirely different frameworks/versions* (even different languages) with zero interop tooling.
- Simplest security boundary — useful when embedding a genuinely untrusted or highly regulated third-party widget (e.g., a payment iframe from a PCI-compliant vendor).

Cons

- Poor UX: independent scrollbars, awkward resizing, no shared browser history/back-button behavior without extra plumbing, degraded accessibility (screen readers, focus management across frame boundaries), harder responsive layout.
- Cross-frame communication is clunky — `postMessage` only, no direct DOM/JS access.

- SEO and deep-linking are harder.
- Duplicate everything: each iframe loads its own framework, fonts, design-system CSS — heavier network/memory cost than any other approach.

Iframes are the right call specifically for hard security/compliance isolation (e.g., embedding a partner's checkout widget), not for general-purpose internal team boundaries.

D. Web Components (Custom Elements)

Each micro frontend is compiled to a **Custom Element** (e.g., `<billing-widget>`) using Angular Elements (`@angular/elements`), and the shell simply drops `<billing-widget>` into its template like any HTML tag. The framework used to build the custom element is irrelevant to the consumer — a Custom Element is a browser-native contract.

Pros

- Framework-agnostic composition — a React shell can host an Angular-built web component and vice versa.
- Natural DOM-level API: attributes/properties in, custom events out — good encapsulation story.
- Shadow DOM (optional) gives real CSS encapsulation (scoped styles, no leakage either direction) without hand-rolled naming conventions.
- No special bundler plugin required (unlike Module Federation) — plain script tags or ES modules work.

Cons

- Each web component typically bundles its *own* copy of Angular + Zone.js (since Custom Elements don't have Module Federation's shared-dependency resolution baked in) unless you engineer sharing yourself — bundle bloat.
- Change detection/Zone.js interplay across multiple Angular instances in one page can be subtle (each has its own zone unless explicitly configured).
- Passing complex data (not just strings) through attributes requires using DOM properties directly or event payloads — routing, lazy-loading, and deep integration (e.g., shared Router) are harder than with Module Federation.

E. Mention: single-spa

`single-spa` is a meta-framework/orchestration library specifically built for micro frontends: it doesn't replace Module Federation, web components, or iframes — it sits above them, providing lifecycle hooks (`bootstrap`, `mount`, `unmount`) so multiple framework applications (Angular, React, Vue, even multiple versions of each) can coexist on the same page, be routed by path/hash, and be mounted/unmounted as the user navigates. Many real-world stacks combine single-spa (orchestration/routing) with Module Federation (code loading/sharing) — they're complementary, not competing, choices. Angular has an official `single-spa-angular` adapter.

2.3 Shared Dependency Management

Across any run-time integration approach, the central technical problem is: **how many copies of Angular/RxJS/Zone.js end up in the browser, and do they interoperate?**

- **Module Federation** `shared config`: each app (host and remotes) declares which packages are shared, with `singleton: true` for packages that *must* have exactly one instance in the page (Angular itself,

Zone.js, often RxJS since Observables from one copy aren't `instanceof` compatible with another copy's operators in some edge cases). `strictVersion` and `requiredVersion` control whether a version mismatch is a hard error or a warning-and-fallback.

- **Semver range negotiation:** Module Federation resolves the shared scope by picking, among all loaded apps, the highest version satisfying every app's declared range; if ranges are incompatible, either duplicate instances load (if `singleton: false`) or the app throws (if `singleton: true` and `strictVersion: true`).
- **Trade-off:** the freedom to deploy independently is fundamentally in tension with tight coupling on framework version — the more you dedupe, the more you implicitly require all teams to stay close to the same major version.

2.4 Cross-Micro-Frontend Communication

Micro frontends need to talk to each other without becoming tightly coupled (that would defeat the point). Three common patterns:

1. **Custom Events (DOM CustomEvent):** one MFE dispatches a `CustomEvent` on `window` or a shared DOM node; others add `window.addEventListener`. This is the loosest coupling — publishers don't know or care who's listening, and it works regardless of which framework built either side (it's a browser primitive). Downside: stringly-typed contracts (event name + payload shape) with no compile-time safety unless you hand-maintain a shared type-only package.
2. **Shared State (event bus / shared service / observable store):** a small shared library (could itself be published as an npm package or exposed as a Module Federation shared singleton) exposes a `BehaviorSubject`-backed store or pub/sub bus that every MFE imports. Stronger typing, but couples every MFE to that shared library's version — a variant of the build-time-integration trade-off, scoped down to just the communication layer.
3. **URL / Routing as state:** encode cross-MFE state in the URL (query params, path segments) so navigation itself is the communication channel. Highly decoupled and gives users shareable/bookmarkable links + working back-button, but only suits state that's meaningful to represent in a URL (not, say, "is this specific modal open with this in-memory draft").
4. (Less common) **postMessage:** required for iframe-isolated MFEs since they don't share a JS realm; same idea as custom events but across frame boundaries with structured-clone payloads.

Rule of thumb used in interviews: prefer the *loosest* coupling that still gets the job done — URL for navigational/shareable state, custom events for one-off notifications ("item added to cart"), shared state only for genuinely shared, frequently-updated cross-cutting data (e.g., current authenticated user).

2.5 Versioning and Independent Deployability

The entire point of micro frontends is undermined if "independent deployability" is only nominal. Practical requirements:

- Each MFE has its own repo (or its own project in a monorepo) and its own CI/CD pipeline that can ship to production without any other team's approval or rebuild.
- The shell resolves remotes **dynamically** (fetch a manifest of remote URLs at runtime, e.g. from a JSON config endpoint) rather than hardcoding remote URLs/versions at the shell's build time — otherwise every remote release still needs a shell redeploy, collapsing back into build-time integration.
- Contract versioning: exposed modules/components and event payloads are a *public API* between teams

and need explicit versioning/deprecation discipline (semver on the exposed surface, not just the internal implementation) — breaking an event payload shape without warning breaks every consumer at runtime with no compiler to catch it.

- Backward/forward compatibility windows: because host and remotes deploy independently, for some window of time an old host may talk to a new remote (or vice versa) — contracts must tolerate at least N-1 compatibility.

3. Code Examples

3.1 Shell (host) consuming a remote via Module Federation

```
// shell-app/webpack.config.js (via @angular-architects/module-federation)
const { withModuleFederationPlugin, shareAll } = require('@angular-architects/module-federation/webpack');

module.exports = withModuleFederationPlugin({
  // Statically-known remotes can be listed here; for true independent
  // deployability, prefer loading remote URLs dynamically at runtime instead.
  remotes: {
    // key: how the host refers to it; value: name@remoteEntry.js URL
    'billingMfe': 'billingMfe@https://billing.example.com/remoteEntry.js',
  },

  shared: {
    ...shareAll({
      singleton: true,          // exactly one instance in the browser
      strictVersion: true,      // throw (don't silently duplicate) on mismatch
      requiredVersion: 'auto' // infer from this app's package.json
    }),
  },
});
```

```
// shell-app/src/app/app.routes.ts
import { Routes } from '@angular/router';
import { loadRemoteModule } from '@angular-architects/module-federation';

export const routes: Routes = [
  {
    path: 'billing',
    loadChildren: () =>
      loadRemoteModule({
        type: 'module',
        remoteEntry: 'https://billing.example.com/remoteEntry.js',
        exposedModule: './Routes', // matches remote's `exposes` key
      }).then((m) => m.BILLING_ROUTES),
  },
];
```

```
// billing-mfe/webpack.config.js – the remote side
```

```
const { withModuleFederationPlugin, shareAll } = require('@angular-architects/module-federation/webpack');

module.exports = withModuleFederationPlugin({
  name: 'billingMfe',

  exposes: {
    // Public surface this remote offers to any host
    './Routes': './src/app/billing.routes.ts',
    './Invoicewidget': './src/app/invoice-widget/invoice-widget.component.ts',
  },

  shared: {
    ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' }),
  },
});
```

```
// shell-app: dynamically resolving the remote's location at runtime
// (this is what makes deployment truly independent – no rebuild of the
// shell is needed when billing.example.com ships a new remoteEntry.js)
import { loadRemoteModule, setRemoteDefinitions } from '@angular-architects/module-federation';

fetch('/assets/mf.manifest.json') // e.g. { "billingMfe":
  "https://billing.example.com/remoteEntry.js" }
  .then((res) => res.json())
  .then((definitions) => setRemoteDefinitions(definitions))
  .then(() => bootstrapApplication(AppComponent, appConfig));
```

3.2 Cross-MFE communication via custom events

```
// shared/mfe-events.ts – a tiny, type-only contract both sides agree on
// (published as a lightweight package or just duplicated by convention;
// note it carries NO runtime code, only types + event name constants,
// so it doesn't force framework-version coupling)
export const CART_ITEM_ADDED = 'app:cart-item-added';

export interface CartItemAddedDetail {
  sku: string;
  quantity: number;
  addedBy: 'catalog-mfe' | 'search-mfe';
}
```

```
// catalog-mfe: publisher – knows nothing about who's listening
import { Component } from '@angular/core';
import { CART_ITEM_ADDED, CartItemAddedDetail } from '../shared/mfe-events';

@Component({
  selector: 'app-product-card',
  standalone: true,
  template: `<button (click)="addToCart()">Add to cart</button>`,
```

```

})
export class ProductCardComponent {
  sku = 'SKU-1234';

  addToCart(): void {
    const detail: CartItemAddedDetail = {
      sku: this.sku,
      quantity: 1,
      addedBy: 'catalog-mfe',
    };

    // Dispatched on `window` so it crosses MFE boundaries regardless of
    // which Angular zone/injector tree produced it.
    window.dispatchEvent(new CustomEvent<CartItemAddedDetail>(CART_ITEM_ADDED, { detail }));
  }
}

```

```

// shell-app / cart-mfe: subscriber – decoupled, no reference to catalog-mfe
import { Injectable, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import { CART_ITEM_ADDED, CartItemAddedDetail } from '../shared/mfe-events';

@Injectable({ providedIn: 'root' })
export class CartEventBridge implements OnDestroy {
  readonly itemAdded$ = new Subject<CartItemAddedDetail>();

  private readonly listener = (evt: Event) => {
    const custom = evt as CustomEvent<CartItemAddedDetail>;
    // NgZone.run may be needed if the dispatching MFE runs outside this
    // Angular instance's zone – otherwise change detection may not fire.
    this.itemAdded$.next(custom.detail);
  };

  constructor() {
    window.addEventListener(CART_ITEM_ADDED, this.listener);
  }

  ngOnDestroy(): void {
    window.removeEventListener(CART_ITEM_ADDED, this.listener);
  }
}

```

4. Internal Working

How Module Federation resolves remotes at runtime. Each federated build emits, alongside its normal bundle, a small `remoteEntry.js` file. This file is *not* the whole application — it's a manifest-plus-loader: it registers the remote's name into a global container object attached to `window` (historically `window[remoteName]`, using the webpack "container" runtime), and exposes a `get(moduleName)` function that, when called, dynamically imports the requested chunk on demand and returns a factory for it. When the host calls `loadRemoteModule(...)`, three things happen in sequence:

1. **Script injection:** the host injects a `<script>` tag for the remote's `remoteEntry.js` URL (or dynamically `import()`s it, for the newer "Module Federation via native ESM" flavor), and waits for it to register itself on the global container.
2. **Shared-scope negotiation:** before actually instantiating any exposed module, both host and remote populate a shared runtime object called the **share scope**. Each side lists, for every dependency it marked `shared`, the version it was built against and the actual module factory. When the remote's code goes to `import '@angular/core'`, the Module Federation runtime intercepts that import and checks the *shared scope* first: if a compatible version is already registered there (typically because the host loaded first), it reuses that exact same module instance instead of pulling in the remote's own bundled copy. This check happens through generated wrapper code (an "init" step, `__webpack_require__.federation / initializeSharing`) that runs before any application code executes.
3. **Module instantiation:** once sharing is resolved, the container's `get(moduleName)` factory is invoked, producing the actual exposed component/module class, which the host then treats like any dynamically imported module (e.g., feeds it to `loadChildren` or instantiates the component).

How singleton shared deps like Angular core get deduplicated. The `singleton: true` flag doesn't change *what* is bundled (the remote's build still may contain its own copy of `@angular/core` as a fallback) — it changes what happens *at runtime resolution time*: the shared-scope negotiation logic will actively search for an already-initialized singleton entry across all federated bundles that have loaded so far, and if one exists (and satisfies the requested semver range, or unconditionally if `strictVersion: false`), every subsequent consumer is pointed at that *same object reference* rather than instantiating their own copy. This matters enormously for Angular specifically because things like dependency injection tokens, `NgModule` metadata registries, and Zone.js's monkey-patched global APIs (`setTimeout`, `Promise`, DOM event listeners) are only correctly shared if there is genuinely one copy in memory — two separate Zone.js instances would each patch the globals independently, causing double-invocation or missed change-detection cycles. If versions are incompatible and `strictVersion: true`, the runtime throws at load time rather than silently running two Angular instances side by side (which is a valid but explicit fallback with `singleton: false`).

5. Edge Cases & Gotchas

- **Version mismatches between micro frontends:** with `singleton: true`, `strictVersion: true`, an incompatible Angular/RxJS version between host and remote throws a runtime error (often surfacing as a cryptic "Unsatisfied version" console error) *only when the user navigates to that remote* — CI at build time cannot catch this because the two apps were never compiled together. Mitigation: contract/version testing in a shared pipeline stage, or a staging environment that deploys all MFEs together before each production release.
- **CSS leaking across boundaries:** neither Module Federation nor npm-package integration provide any CSS isolation — global stylesheets, unscoped class names (`.card`, `.header`), and even Angular's default view encapsulation (`Emulated`) only scopes *within* one Angular application instance, not across federated remotes loaded into the same DOM tree. Practical mitigations: enforce a CSS naming convention/prefix per MFE (e.g., BEM with an MFE-specific prefix), use `ViewEncapsulation.ShadowDom` for genuine style isolation (at the cost of losing some global theming capability), or wrap each MFE's root in a Shadow DOM boundary via a custom element wrapper.
- **Duplicate Zone.js instances:** if `zone.js` isn't correctly declared as a shared singleton (or if one MFE was built with a Zone.js version incompatible with another's), you can end up with two zones patching the same global APIs. Symptoms include change detection firing twice for one event, or firing in the

wrong MFE's Angular instance, or silently *not* firing at all because an event handler was registered under a zone the running MFE isn't listening to. Zone.js is a common source of "works in isolation, breaks when composed" bugs in Module Federation setups, and is one reason some newer architectures explore zoneless Angular (`provideExperimentalZonelessChangeDetection` and later stable zoneless APIs) partly *because* it removes this entire class of composition bug.

- **Shared auth/session token synchronization:** each MFE typically cannot independently manage authentication if the user is meant to experience one login. Common approaches: a shared, `httpOnly` session cookie (works automatically across same-origin MFEs, requires care/CORS config across subdomains), or a shared auth service exposed as a Module Federation singleton that all MFEs consume for the current token, with a broadcast mechanism (`BroadcastChannel` API, or a custom event) to notify all loaded MFEs when the token refreshes or the user logs out, so an MFE holding a stale token doesn't keep making requests that silently 401. A frequent real bug: MFE A refreshes the token and updates its own in-memory copy, but MFE B (bundled and loaded separately) still holds the old token in its own service instance because the "shared" service wasn't actually deduplicated (e.g., `singleton` wasn't set, or the two MFEs were built against incompatible versions of the shared auth library so Module Federation fell back to loading two copies).
- **Independent deploys breaking contracts silently:** because there's no shared compiler, a remote can change an exposed component's `@Input()` name or an event's payload shape and ship it to production; the host only discovers the break at runtime (often only when a specific code path executes), not at any build step. This is the single biggest argument for automated contract tests / consumer-driven contract testing between host and remotes, and for keeping exposed surfaces intentionally minimal and stable.

6. Interview Questions & Answers

Q1. What problem do micro frontends solve that a well-organized monolith with lazy-loaded modules doesn't?

A: Lazy-loaded Angular modules solve bundle-size/code-splitting within a single deployable unit, but every team still shares one repo, one build, one release train — a bug or a slow CI run from one team blocks everyone's release. Micro frontends solve *organizational* coupling: independent deployability, independent tech-stack/version choices per team, and clearer ownership boundaries mapped to team boundaries (Conway's Law). If your real pain is "one big bundle," you likely don't need micro frontends — you need better code-splitting/monorepo tooling.

Q2. Compare build-time integration (npm packages) with Module Federation. When would you pick each?

A: Build-time integration (publishing MFEs as versioned npm libraries the shell imports) gives full compile-time type safety, one optimized bundle, no runtime resolution risk — but the shell must rebuild/redeploy for every MFE change, so it's not truly independently deployable. Module Federation loads separately-built, separately-deployed bundles at runtime and can dedupe shared singletons — giving real independent deployability — at the cost of runtime-only failure modes (version mismatches, network dependency on the remote being up) and heavier tooling/debugging. Pick build-time integration when you want most of the ownership benefits without deployment complexity and can tolerate coordinated releases; pick Module Federation when independent production deploys per team is a hard requirement.

Interviewer intent: Checking whether the candidate understands that "micro frontends" isn't one technique — the choice is a deliberate trade-off between safety/simplicity and deployment autonomy, not a default "use Module Federation" answer.

Q3. Why would you choose iframes over Module Federation for a given micro frontend?

A: Iframes are the right choice specifically when you need hard isolation guarantees — a genuinely separate JS realm and DOM, so a broken or actively malicious third-party widget (e.g., an embedded partner payment form) cannot read/mutate the host's global state, DOM, or cookies, and cannot crash the host's JS context. Module Federation provides no such isolation — it's designed for cooperative, trusted teams sharing one JS realm. The cost is UX degradation (scroll, resize, focus, accessibility) and total duplication of framework/asset weight, so iframes are a deliberate security/compliance trade-off, not a general-purpose composition mechanism.

Q4. What is `singleton: true` in a Module Federation `shared` config actually doing, mechanically?

A: It doesn't change what gets bundled — a remote may still ship its own copy of the dependency as a fallback. It changes runtime resolution: before instantiating any exposed module, Module Federation's shared-scope negotiation checks whether a compatible instance of that dependency is already registered globally (from the host or an earlier-loaded remote); if so, every consumer is pointed at that same module instance rather than each getting its own. Combined with `strictVersion: true`, an incompatible version among the "singleton" candidates throws at load time instead of silently duplicating the dependency.

Interviewer intent: Distinguishing candidates who've only used the `shared` config from copy-pasted boilerplate versus those who understand it's a runtime negotiation, not a build-time dedupe.

Q5. Why does having two copies of Zone.js loaded cause bugs, concretely?

A: Zone.js works by monkey-patching global async APIs (`setTimeout`, `Promise.then`, `DOM.addEventListener`, `XHR`, etc.) so Angular can detect when async work completes and knows to run change detection. If two independent Zone.js instances patch the same globals, each maintains its own notion of "the current zone" and its own patched references — an async callback scheduled under one Zone.js instance's patched `setTimeout` may not notify the *other* instance's Angular application that anything happened, so that application's change detection never runs (stale UI), or work runs twice if both instances end up wrapping the same callback. The fix is ensuring Zone.js is declared as a `singleton: true` shared dependency (or moving to zoneless Angular, which sidesteps the whole class of problem by not depending on monkeypatched globals for change detection).

Q6. How would you handle CSS isolation across independently built micro frontends?

A: No integration approach (Module Federation, web components without Shadow DOM, npm packages) provides CSS isolation automatically — Angular's `ViewEncapsulation.Emulated` only scopes within one Angular app instance, not across federated bundles sharing a DOM. Options, roughly in order of isolation strength: (1) naming convention/prefixing discipline (BEM-with-MFE-prefix) — cheapest but relies on discipline, not enforcement; (2) CSS Modules or scoped build-time class hashing, if your build tooling supports it across MFEs; (3) `ViewEncapsulation.ShadowDom` (or wrapping the MFE root in a native Custom Element with Shadow DOM) — true browser-enforced isolation, at the cost of harder global theming (design tokens must cross the shadow boundary via CSS custom properties, which do pierce shadow DOM). For genuinely untrusted content, iframes remain the strongest isolation.

Q7. Two micro frontends need to know "is the user logged in and who are they" — how do you avoid duplicating auth logic and avoid stale-token bugs?

A: Centralize the source of truth: either (a) a shared, `httpOnly`, same-site session cookie that every MFE's HTTP layer automatically includes (works well for same-origin/subdomain deployments, needs care with CORS/SameSite for cross-origin remotes), or (b) a single shared auth service exposed as a Module Federation `singleton` shared dependency, so there is genuinely one in-memory token/user-state object regardless of how many MFEs reference it. Either way, you need an explicit invalidation/refresh broadcast — a `BroadcastChannel` message or custom event fired on token refresh/logout — so MFEs that cached a token

locally (rather than reading the shared service live) get notified rather than silently using a stale token until their next unrelated re-render.

Q8. What's the actual risk if micro frontend contracts (exposed components' inputs/outputs, or custom event payloads) aren't versioned carefully?

A: Because host and remotes are compiled and deployed independently, there is no compiler step that catches a breaking change to an MFE's public surface — e.g., a remote renaming an `@Input()` or changing an event's payload shape ships straight to production, and the break only manifests at runtime, potentially only on a specific route a QA pass didn't hit. This is why exposed surfaces need explicit semver discipline and ideally consumer-driven contract tests (verifying the host's expectations against the remote's actual exposed API) run in CI *before* a remote's release, not caught after the fact in production.

Interviewer intent: Probing whether the candidate has actually operated a Module Federation setup in production versus only having read about the happy path — contract drift is the most common real-world Module Federation incident.

Q9. Custom events vs. a shared state service vs. URL for cross-MFE communication — how do you choose?

A: Prefer the loosest coupling that satisfies the requirement. URL/query-params/path segments are best for anything that should be shareable, bookmarkable, or survive a refresh (e.g., "which tab is active," "which record is selected") and require zero shared code between MFEs. Custom events (DOM `CustomEvent` on `window`) suit one-off, fire-and-forget notifications ("item added to cart") where the publisher shouldn't need to know who, if anyone, is listening — but the contract (event name + payload) is stringly typed unless you maintain a small shared types-only package. A shared state service (a singleton store/event bus) is justified only for genuinely cross-cutting, frequently-read state (current user, feature flags, cart contents needed by many consumers) — it's the most powerful option but also creates the tightest coupling, since every consumer now depends on that shared library's shape and version.

Q10. How does the shell resolve *where* a remote lives, and why does this matter for independent deployability?

A: If the shell's `ModuleFederationPlugin/withModuleFederationPlugin` config hardcodes `remotes: { billingMfe: 'billingMfe@https://billing.example.com/remoteEntry.js' }` at build time, the URL is fixed at the shell's compile time — fine for the URL staying stable across a remote's releases (the remote can still redeploy new code at that same URL independently), but if you ever need to change environments, add a canary URL, or point different users at different remote versions, you're back to needing a shell rebuild. Truly dynamic setups instead fetch a small manifest (a JSON file mapping remote names to current URLs) at runtime before bootstrapping, via `setRemoteDefinitions`, so the shell's own build artifact never has to change when a remote ships — only the manifest does, which can be its own tiny, instantly-deployable artifact.

Q11. What are the main sources of duplicated bytes-over-the-wire in a Module Federation setup, and how do you minimize them?

A: Duplication happens when `shared` deps aren't actually deduplicated at runtime: mismatched `requiredVersion` ranges causing `singleton: false` fallback behavior (each app loads its own copy), forgetting to mark a heavy library (e.g., a large UI kit, moment/date-fns, RxJS) as shared at all, or Angular version drift between host and remotes wide enough that `strictVersion` forces separate copies. Minimizing this requires an org-wide policy on how close to lockstep shared dependency versions must stay (e.g., "all MFEs must be within the same Angular major, updated within N weeks of each other"), consistent `shareAll`/explicit `shared` configuration across every federated app (not just the host), and monitoring bundle/network reports post-deploy to catch when a dependency silently stopped being deduped.

Q12. Angular Elements/web components vs. Module Federation for composing a page from multiple frameworks (e.g., one section built in React, another in Angular) — which fits better and why?

A: Web Components (via Angular Elements on the Angular side) are the natural choice when the composing shell isn't itself Angular, or when different sections are built in genuinely different frameworks — Custom Elements are a browser-native contract that any framework can both produce and consume, with no build-tool-specific integration required. Module Federation is framework-agnostic in principle but its tooling and shared-dependency story are most mature/ergonomic when all federated apps share the same bundler ecosystem (webpack-family) and ideally similar dependency graphs — mixing React and Angular via Module Federation is possible but you get none of the "shared singleton" benefit across frameworks (there's no shared React/Angular to dedupe), so you're mostly using it as a fancy dynamic- `import()` mechanism rather than getting its headline dependency-sharing benefit. In short: cross-framework composition favors Web Components; same-framework, multi-team composition with real dependency sharing favors Module Federation.

Interviewer intent: Testing whether the candidate conflates "Module Federation" with "micro frontends" as if they were synonyms, versus understanding Module Federation is one tool with a specific sweet spot (federating apps on the same bundler/framework family).

Q13. What is single-spa, and how does it relate to Module Federation?

A: `single-spa` is an orchestration meta-framework for micro frontends: it defines a lifecycle contract (`bootstrap`, `mount`, `unmount`) that lets multiple independently built applications — potentially in different frameworks or different versions of the same framework — coexist on one page, with single-spa's router activating/deactivating each based on the current path/hash. It solves *orchestration and lifecycle*, not code delivery or dependency sharing — those are what Module Federation (or npm packages, or plain script tags) solve. In practice the two are frequently combined: Module Federation handles fetching and dependency-deduplicated loading of each MFE's code, while single-spa handles deciding which MFE(s) should be mounted for the current route and manages their mount/unmount lifecycle cleanly (important for preventing memory leaks and duplicate event listeners when a user navigates away from and back to an MFE repeatedly).

7. Quick Revision Cheat Sheet

- **Why micro frontends:** organizational scaling — independent team deploys, independent tech choices, vertical ownership — not a default architecture; adopt only when multi-team coordination pain is real.
- **Build-time (npm packages):** full type safety, one bundle, but shell must rebuild per MFE release — not truly independently deployable.
- **Module Federation (run-time):** separately built/deployed bundles loaded at runtime; `remoteEntry.js` registers a container; `shared` config (`singleton`, `strictVersion`, `requiredVersion`) controls dependency dedup; enables true independent deploys at the cost of runtime-only failure modes.
- **iframes:** maximum isolation (separate JS realm/DOM/CSS), best for untrusted/compliance-sensitive embeds; worst UX/perf overhead; `postMessage` only for communication.
- **Web Components (Angular Elements):** framework-agnostic, browser-native composition; Shadow DOM gives real CSS isolation; typically no built-in dependency dedup (each ships its own Angular/Zone.js) unless engineered.
- **single-spa:** orchestration/lifecycle layer (`bootstrap`/`mount`/`unmount` + routing across MFEs) — complements Module Federation, doesn't replace it.
- **Shared deps:** `singleton: true` = one runtime instance across all federated apps if versions are compatible; `strictVersion: true` = hard error instead of silent duplication on mismatch.

- **Cross-MFE communication:** URL (shareable/bookmarkable state) > custom events (loose, fire-and-forget notifications) > shared state service (strongest coupling, use only for real cross-cutting state).
- **Versioning/independent deployability:** only real if remote URLs are resolved dynamically at runtime (manifest fetch), not hardcoded at the shell's build time; exposed component APIs and event payloads are a public contract needing semver + contract tests.
- **Gotchas:** version mismatches surface only at runtime on the affected route; CSS leaks by default (no isolation from Module Federation/npm approaches); duplicate Zone.js causes missed/duplicated change detection; auth tokens must be a shared singleton with an explicit refresh/logout broadcast (`BroadcastChannel` / custom event) to avoid stale-token bugs in MFEs that cached a copy.

Created By - Durgesh Singh